

极致成本控制： 混合模型路由策略与 API 聚合

用一份配置文件驯服十种大模型，让旗舰级智能按需上阵、按量计费

彭靖田 | 谷歌 AI 开发者专家 · 上海载极数据科技 CEO

本节 课程地图

1 Token 经济学与成本真相

你的 Agent 每月到底要花多少钱？

2 模型供应商解耦与模型配置

一份配置文件管理所有大模型

3 Fallback 容错链与成本防线

用不了的时候自动换，花超了自动停

4 实战：端到端混合路由配置

亲手搭建三级成本防线，让它断给你看

5 总结与思考

本节核心收获与课后作业

第一章

Token 经济学与成本真相

你的 Agent 每月到底要花多少钱？

Polymarket 赔率监控 Agent 真正跑起来，
每 5 分钟一次定时任务，
一个月 Token 花多少钱？

上节课结尾留下的问题，今天我们来尝试计算下。

Token 到底是什么？一次 Agent 对话的真实账单

Token $\approx$ 大模型的"计费单位", 中文约 1 字 $\approx$ 1.5–2 个 Token

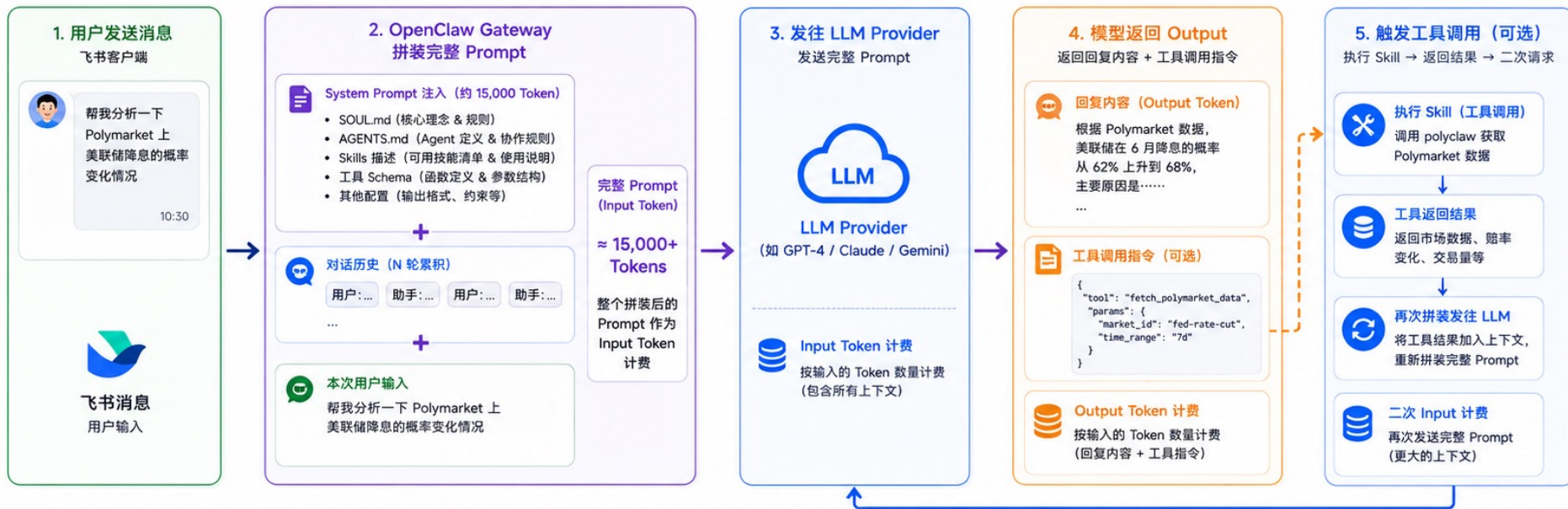
一次 Agent 对话的 Token 构成远比你想象的复杂

构成项	大致占比	说明
System Prompt (SOUL.md + Skills + 工具定义)	40–50%	每次对话都要重发, 是最大隐性成本
对话历史 (越长越贵)	20–30%	不做 Compaction 会累积爆炸
用户输入 + 模型输出	15–25%	你以为的"全部成本", 其实只是冰山一角
工具调用结果 (搜索/抓取返回)	5–15%	取决于 Skill 返回内容的长度

System Prompt 和对话历史占了 60–80%, 这才是成本大头

Token 流向全景：一条消息的真实旅程

从用户消息到最终回复，每一步都在消耗 Token



真实的 Token 流向



为什么每轮都要重发完整上下文？

- ✓ LLM 无状态，无法记住历史对话
- ✓ 每次请求都需要完整的上下文来理解语境
- ✓ 包括 System Prompt、历史对话、用户输入、工具结果等
- ✓ 上下文越长，消耗的 Input Token 越多

关键事实

一次“简单问答”实际可能产生 2-3 轮 API 调用，每轮都重发完整上下文



优化建议：合理管理对话历史长度、精简 System Prompt、使用结构化数据、选择合适的模型上下文窗口

Polymarket 监控 Agent：一个月烧多少钱？

¥200/月

Coding Plan Pro 包月封顶

¥930+/月

GPT-5.4 按量付费

¥5,000+

上下文失控时

计算依据

- 频率：每 5 分钟一次，每月 ~8,640 次；每次约 3,000 Input + 500 Output Token
- GPT-5.4 (\$2.50/\$15.00 per M)：约 \$129/月 ≈ ¥930/月
- Claude Opus 4.6 (\$5.00/\$25.00 per M)：约 \$238/月 ≈ ¥1,714/月
- 火山引擎 Coding Plan Pro：¥200/月包月封顶，足以覆盖每 5 分钟一次的巡检频率
- 不做 Compaction、上下文膨胀 5-10 倍：GPT-5.4 可达 ¥4,000+，Opus 可达 ¥5,000+

同一个 Agent、同一个任务，成本差异可达 25 倍

两条路线的本质差异

按量付费 (Hotai 中转站)

用多少付多少

精确到每个 Token

适合：低频高价值任务

复杂推理、关键决策

风险：不设上限可能爆预算

课程使用：GPT-5.4 + Claude Opus 4.6

包月订阅 (火山引擎 Coding Plan)

固定月费封顶

Lite ¥40 / Pro ¥200, 推荐 Pro

适合：高频日常任务

心跳巡检、日报生成、常规问答

超频次限流，不产生额外费用

课程使用：DeepSeek V3.2 / GLM-5 / Kimi K2.6

旗舰对决：GPT-5.4 vs Claude Opus 4.6

维度	GPT-5.4	Claude Opus 4.6
定价 (Input / Output)	\$2.50 / \$15.00	\$5.00 / \$25.00
上下文窗口	1M tokens	1M tokens
单次同等任务成本	基准 1×	约 1.7×
指令遵循	强，偶有"自由发挥"	极强，严格按指令执行
工具调用 (Tool Use)	稳定，生态成熟	极稳定，结构化输出精度更高
长文本理解	1M 窗口实测可用	1M 窗口，长文本保持度优秀
代码生成	全栈强，前端尤其突出	系统设计和重构能力突出
中文能力	良好	优秀，中文语感更自然
多模态	图文音视频全支持	图文，视频支持有限
接入方式 (本课程)	Hotai 中转站	Hotai 中转站

ROI 分析：什么时候值得为 Opus 多花 70%?

选 GPT-5.4 的场景（性价比优先）

通用问答和日常对话

创意内容生成（文案、营销、头脑风暴）

多模态任务

图片理解、音频处理

前端代码生成

ROI：便宜 40% 的成本差更值得

选 Claude Opus 4.6 的场景（精度优先）

复杂多步工具调用链

Skill 嵌套执行、结构化数据提取

长文档理解与精准摘要

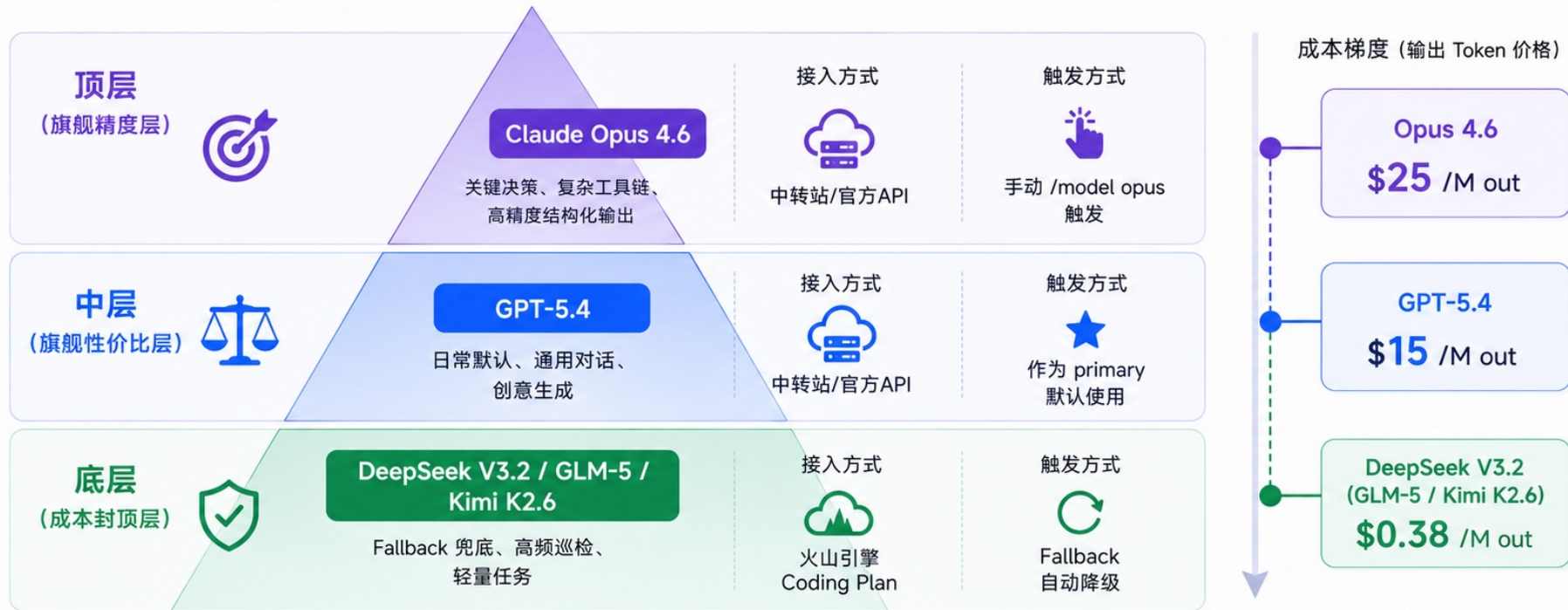
JSON / 表格输出精度

严格遵循指令的自动化流程

ROI：多花 70%，减少 50%+ 返工

大模型完整三层矩阵

精度、性价比、成本封顶的最优解组合



三层不是“好中差”，而是“精度 - 性价比 - 成本封顶”三种不同的优化目标



顶层追求极致精度
解决最难的问题



中层追求性价比平衡
覆盖 80% 的日常场景



底层锁定成本上限
保证系统可持续运行

灵魂拷问

既然国产模型这么便宜，
为什么不全用国产？

答案：问题不是选谁，而是——什么时候该花这个钱，什么时候不该

第二章

模型供应商解耦与模型配置

一份配置文件管理所有大模型

OpenClaw 的模型配置体系

OpenClaw 是 LLM-Agnostic 的——不绑定任何一家模型提供商

通过 **openclaw.json** 统一管理所有模型

配置文件位置

~/.openclaw/openclaw.json

三个核心配置块

models (Provider 注册 + 模式)、agents.defaults.model (默认模型 + Fallback)、.env (API Key 安全存储)

models.mode: "replace" (本课程使用)

每次启动时 openclaw.json 中的 Provider 配置完整覆盖到 Agent 本地的 models.json，改了配置 /restart 立刻生效

models.mode: "merge" (OpenClaw 默认值)

Agent 本地已有的 apiKey/baseUrl 会被保留，全局改了可能不生效，需要手动清理 Agent 本地文件

内置模型目录始终可用

为某个 Provider 配了 API Key 后，该 Provider 下所有内置模型自动获得认证

openclaw.json 模型配置三层架构

三层职责分离，灵活路由，稳定兜底

运行时覆盖来源



顶层：运行时覆盖层

临时指定，特殊任务换路

session 级 / 任务级

```
// 运行时临时指定，优先级最高，不走 fallback
/model claude-opus-4-6
// Cron 任务
openclaw run job.yaml --model gpt-5-4
// per-agent 配置
agents:
  research:
    model: deepseek-v3-2
```

适用场景



关键决策 / 高精度任务



临时性高优先级任务



特定 Agent 特殊配置

openclaw.json 配置（路由策略）



中层：路由决策层

默认路由，失败走 fallback 链

全局默认策略

```
"agents": {
  "defaults": {
    "model": {
      "primary": "gpt-5-4",
      "fallbacks": [
        "deepseek-v3-2",
        "glm-5",
        "kimi-k2-6"
      ]
    }
  }
}
```

路由行为



优先使用 primary 模型



失败或限流时，按顺序 fallback 到下一个



直到成功或耗尽所有选项

openclaw.json 配置（供应商注册）



底层：Provider 注册层

注册供应商与模型能力

基础能力声明

```
"models": {
  "providers": {
    "anthropic": {
      "baseUrl": "https://api.anthropic.com",
      "apiKey": "${ANTHROPIC_API_KEY}",
      "models": ["claude-opus-4-6", "claude-sonnet-4"]
    },
    "openai": {
      "baseUrl": "https://api.openai.com/v1",
      "apiKey": "${OPENAI_API_KEY}",
      "models": ["gpt-5-4", "gpt-5", "gpt-4o"]
    },
    "volcengine": {
      "baseUrl": "https://ark.cn-beijing.volces.com/api/v3",
      "apiKey": "${VOLCENGINE_API_KEY}",
      "models": ["deepseek-v3-2", "glm-5", "kimi-k2-6"]
    }
  }
}
```

作用



定义每个提供商的 baseUrl, apiKey



声明可用模型列表及能力范围



告诉 OpenClaw 你有哪些供应商

优先级与行为说明

1

顶层：运行时覆盖层

优先级最高

严格模式（不走 Fallback）

只使用指定模型，失败即报错



2

中层：路由决策层

次优先级

Fallback 模式

按 fallbacks 有序列表自动降级



3

底层：Provider 注册层

基础层

能力声明与注册

不参与路由决策

优先级关系

顶层 > 中层 > 底层

从上到下，逐层兜底



总结

底层提供“路的选择” → 中层决定“走哪条路、断了走哪条” → 顶层实现“临时改路”

三层解耦，配置清晰，路由稳定，灵活可控

openclaw.json 的 Provider 配置结构

openclaw.json

```
{
  "models": {
    "mode": "replace",
    "providers": {
      "hotai": {
        "baseUrl": "https://www.hotaitool.net/v1",
        "apiKey": "${HOTAI_API_KEY}",
        "api": "openai-responses",
        "models": [
          { "id": "gpt-5.4", "name": "GPT-5.4" }
        ]
      },
      "volcengine": {
        "baseUrl": ".../api/coding/v3",
        "apiKey": "${ARK_API_KEY}",
        "api": "openai-completions",
        "models": [{
          "id": "ark-code-latest",
          "name": "ark-code-latest",
          "contextWindow": 256000,
          "maxTokens": 32000,
          "input": ["text", "image"]
        }]
      }
    }
  }
}
```

① mode: "replace"

每次启动覆盖 Agent 本地配置，改了就生效

② Hotai

一个 API Key 绑一个模型，这里绑 GPT-5.4

③ Opus 需独立 Provider

需在 hotai 创建独立账号和 API Key

④ 火山引擎智能调度

声明 ark-code-latest，内置国产模型自动可用

⑤ API 协议不同

Hotai: openai-responses / 火山: openai-completions

⑥ API Key 安全隔离

\${VAR} 占位，实际值放 ~/.openclaw/.env

默认模型与 Fallback 链配置

openclaw.json

```
{
  "agents": {
    "defaults": {
      "model": {
        "primary": "hotai/gpt-5.4",
        "fallbacks": [
          "volcengine/ark-code-latest"
        ]
      }
    }
  }
}
```

① primary

日常默认使用的模型格式：provider/model-id

② fallbacks 有序列表

GPT-5.4（旗舰按量）→ ark-code-latest（国产包月智能调度）

③ Opus 本节课暂不配置

需要可以在平台创建新的 API Key 配置为另一个 Provider 后通过/model opus 切换

④ 触发条件

API 报错（timeout / rate limit / 5xx / 额度耗尽），不是“觉得贵了”自动降级

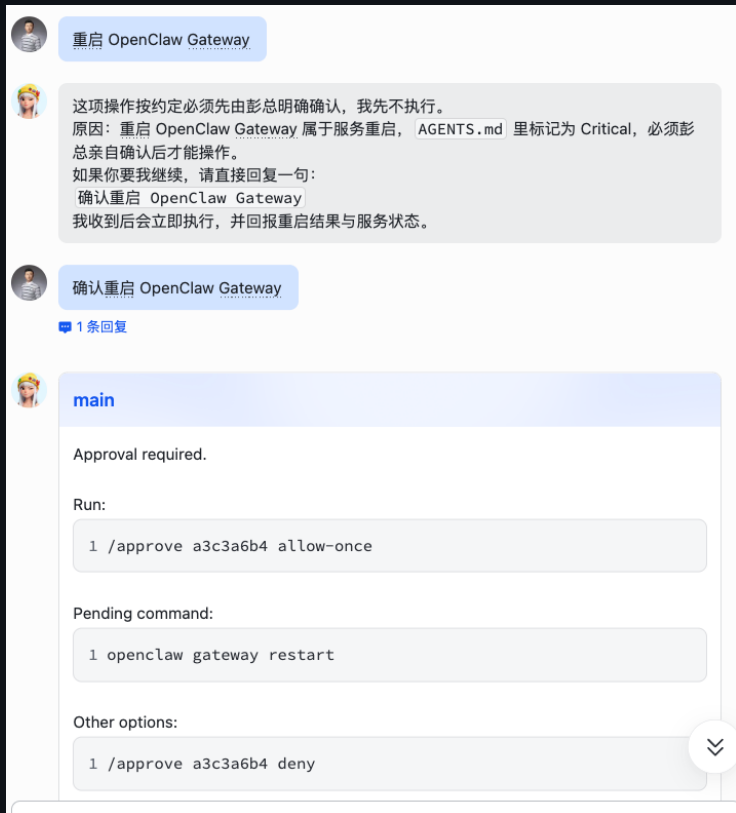
动手：配置双 Provider + Fallback 链

操作指引

- 1 用 VSCode Remote 打开服务器上的 `~/openclaw/openclaw.json`
- 2 添加 hotai 和 volcengine 两个 Provider 配置
- 3 设置 `agents.defaults.model.primary` 和 `fallbacks`
- 4 保存文件后，在飞书中发送重启 OpenClaw Gateway
- 5 在飞书中发送 `/status` 验证配置

预期结果

Agent 回复当前模型状态，显示 primary 指向 hotai/gpt-5.4



验证：飞书中 /status 的正确输出

🧠 Model: hotai/gpt-5.4 ✅

可通过 /model volcengine/ark-code-latest 手动切换到国产智能调度

如果显示 Missing auth 或模型不对，检查 API Key 和 agents.defaults.model.primary



/status



🔧 OpenClaw 2026.4.22 (00bd2cf)
🧠 Model: hotai/gpt-5.4 · 🔑 api-key (models.json)
🔄 Fallbacks: volcengine/ark-code-latest
📊 Tokens: 20k in / 147 out
🗄️ Cache: 7% hit · 1.5k cached, 0 new
📄 Context: 22k/200k (11%) · 🛠️ Compactions: 0
📘 Session: agent:main:main · updated just now
⚙️ Runtime: direct · Runner: pi (embedded) · Think: off · elevated
🚦 Queue: collect (depth 0)

第三章

Fallback 容错链与成本防线

用不了的时候自动换，花超了自动停

三级成本防线架构

(Layered Protection, Cost Control, Stable Service)

防线层级



第一层（模型供应商熔断断）

Hotai 中转站设置配额 / rate limit
超额返回 HTTP 429

这是“硬开关”，花完就断

OpenClaw
请求发送



Hotai
配额/速率控制

已用 85% (上数 \$500)

超额返回
HTTP 429 Error



请求被拒绝

作用

- 在模型供应商侧直接拦截
- 触发 HTTP 429 速率限制
- 这是“硬开关”，花完就断
- 彻底控制成本下限

1

硬熔断
花完就断
不能超



船发条件：收到 402/429 或请求失败



第二层（OpenClaw Fallback 链）

第一层断了 → 自动切换到火山引擎
Coding Plan 国产模型

服务不中断，成本锁定在 ¥200/月包月

OpenClaw
检测到失败

Fallback 链
自动切至备用模型

火山引擎
Coding Plan

火山引擎持续锁定在 ¥200/月包月

请求成功



服务不中断

作用

- Hotai 断了之后自动切换
- 确保服务持续可用
- 成本锁定在 ¥200/月包月
- 避免未知的突发费用

2

Fallback
断了就切
不会断



持续监控：记录消耗，分析趋势，预算异常



第三层（事后审计）

Hotai 后台查看消耗明细
Gateway 日志查看 Fallback 记录

rate limit 触发本身就是“预算用完”的信号



Hotai 后台审计
查看消耗明细

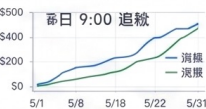


Gateway 日志审计
查看Fallback审计

Log line 2 155 Fallback trigger: "Motenca...
Log line 2 155 Fallback trigger: "Motenca...
Log line 2 155 Fallback trigger: "Motenca...
Log line 2 155 Fallback trigger: "Motenca...
Log line 2 155 Fallback trigger: "Motenca..."



rate limit 触发
本身就是“预算用完”



作用

- 定期审计消耗明细
- Gateway 日志追溯 Fallback
- 明确率限制触发的含义
- 持续优化成本结构

3

审计
查看记录，
看得清



总结

硬熔断管“不能超”

提供商侧硬开关
超额直接拦截



Fallback 管“不会断”

自动切换备用模型
服务持续可用



监控管“看得见”

实时监控与预警
提前发现异常



成本可控
服务稳定

第一层：Hotai 中转站配额设置

Step 1 登录 Hotai 中转站管理后台

Step 2 进入 API Key 管理 → 选择对应 Key → 设置“额度上限”

Step 3 设置一个较低的测试额度（如 \$0.01），用于后续验证

Step 4 确认保存 → 超额时 Hotai 返回错误 → OpenClaw Fallback 接管

生产环境建议设为月预算的 120%（留 buffer），测试环境故意设低来验证 Fallback

第二层：火山引擎 Coding Plan 的"天然熔断"

Coding Plan 是包月制，超过频次限制只会"限流"，不会产生额外费用

Pro ¥200/月（高频场景推荐）

一个 API Key 覆盖 DeepSeek V3.2 / GLM-5 / Kimi K2.6 等所有国产模型

Lite ¥40/月适合轻度使用

超频次限流，不是拒绝

等下一个周期自动恢复，不会产生额外账单

这就是"天然熔断"，成本上限锁死在 ¥200/月

与 Hotai 的本质区别

Hotai：用多少花多少（可能爆账单）

Coding Plan：花多少用多少（成本封顶）

Fallback 的触发机制

Fallback 是 OpenClaw 内置的原生能力，无需写代码

触发条件：API 返回错误（timeout / rate limit / HTTP 5xx / 额度耗尽）

默认 primary → 触发 Fallback 链 ✓

自动跳到 fallbacks 列表的下一个模型，用户无感知

手动 /model 选择 → 不触发 Fallback，直接报错 ✗

严格模式：你特意选了旗舰，如果偷偷换成便宜模型你可能不知道

Cron 任务 → 默认走 Fallback

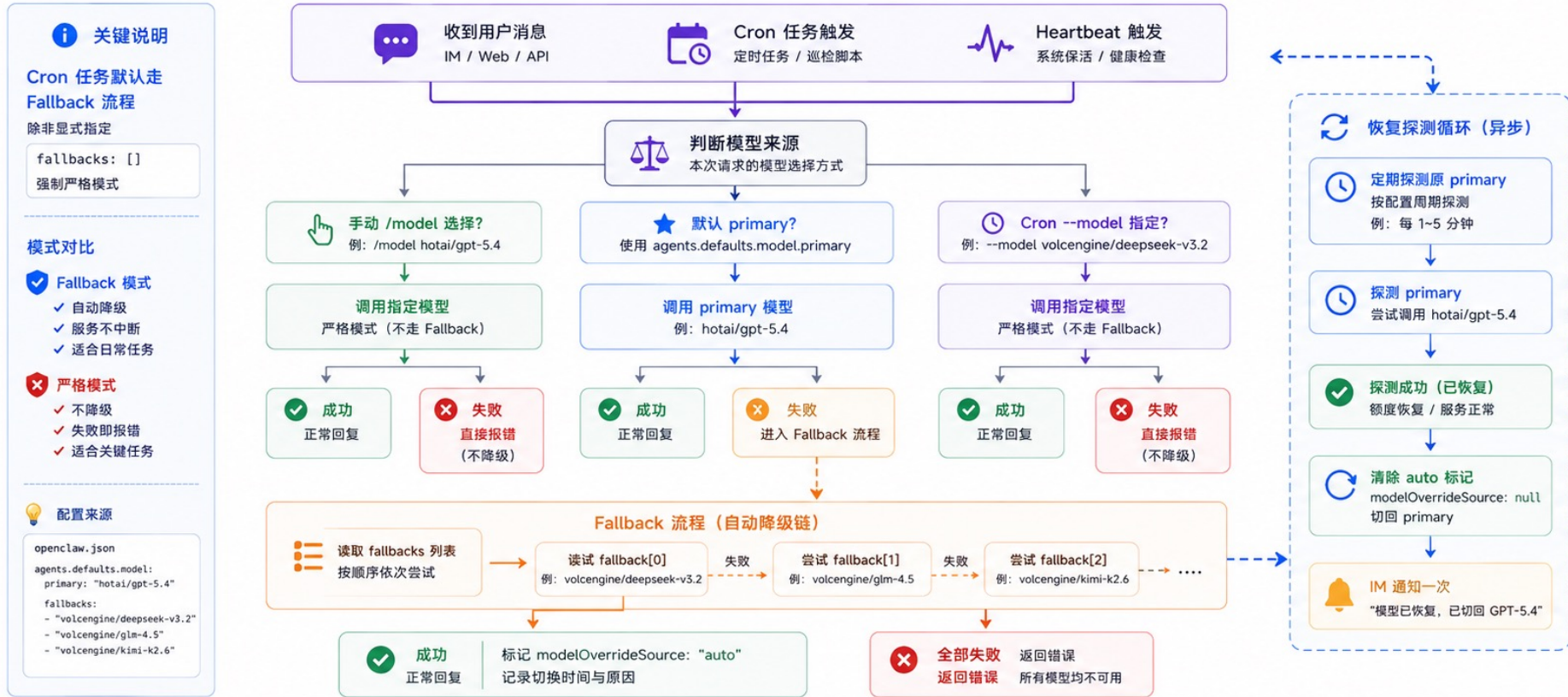
除非显式指定 fallbacks: [] 强制严格模式

恢复机制

OpenClaw 定期探测原 primary，恢复后自动切回，IM 通知一次状态变化

Fallback 链运行时决策流程

智能路由 · 自动降级 · 自动恢复 · 服务不中断



流程总结



手动选择
严格模式, 不降级



默认 / Cron 默认
走 Fallback 链, 服务不中断



自动切换
标记 auto, 透明降级



自动恢复
探测成功, 自动切回



目标: 高可用 + 成本可控
不打扰用户, 稳定最重要

第三层：事后审计与消耗查看

rate limit 触发本身就是"预算用完"的信号，不需要额外的实时监控

Hotai 后台

登录中转站管理后台，查看 API Key 的消耗明细和剩余额度

Gateway 日志

在飞书中让 Agent 查 Gateway 日志，可以看到 429 限流记录和 Fallback 切换记录

回复风格变化（辅助判断）

Fallback 到国产模型后，回复风格、速度、中文表达可能有细微差异

为什么不需要实时监控？

第一层 rate limit 就是你设的预算线，触发了说明预算到了。Fallback 自动接管，成本已经锁定

灵魂拷问

配好了防线，
但你怎么知道它真的能生效？

唯一的验证方法是一一让它真的断一次。接下来我们亲手触发 Fallback。

第四章

实战：端到端混合路由配置

亲手搭建三级成本防线，让它断给你看

实战总览：四步验证混合路由

Step 1 在 Hotai 后台 rate limit 设为极低值 (如 \$0.01)

Step 2 在飞书中发送一条任务，触发 Hotai rate limit，Fallback 静默生效

Step 3 让 Agent 查 Gateway 日志，确认 429 限流记录和 Fallback 切换记录

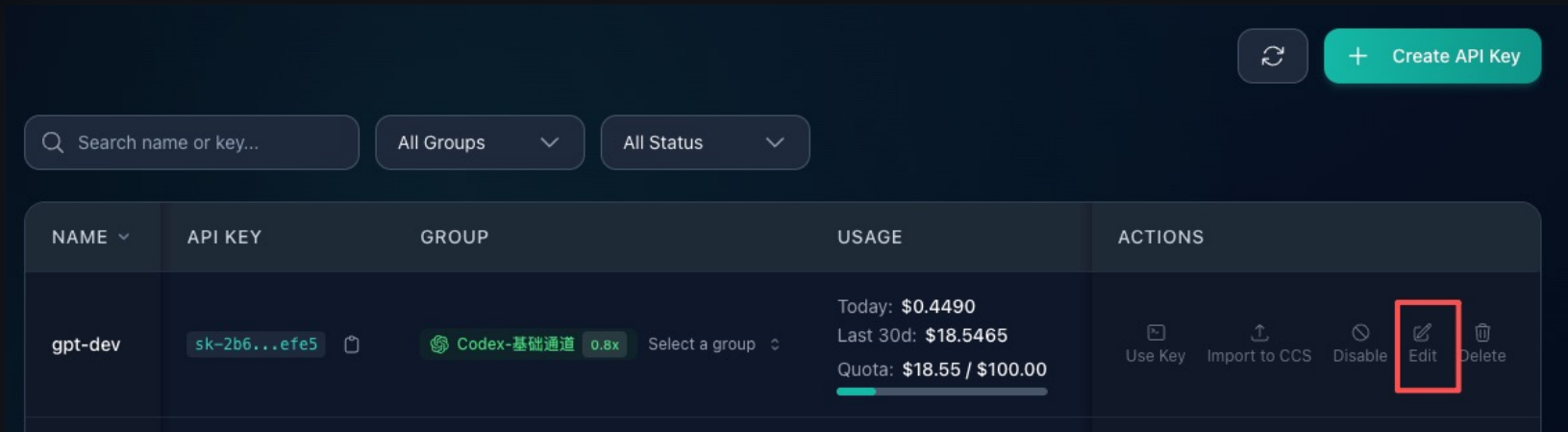
Step 4 恢复 Hotai 额度，等待自动切回 primary，查日志确认恢复

实战

准备：设置 Rate Limit 低配额

操作指引

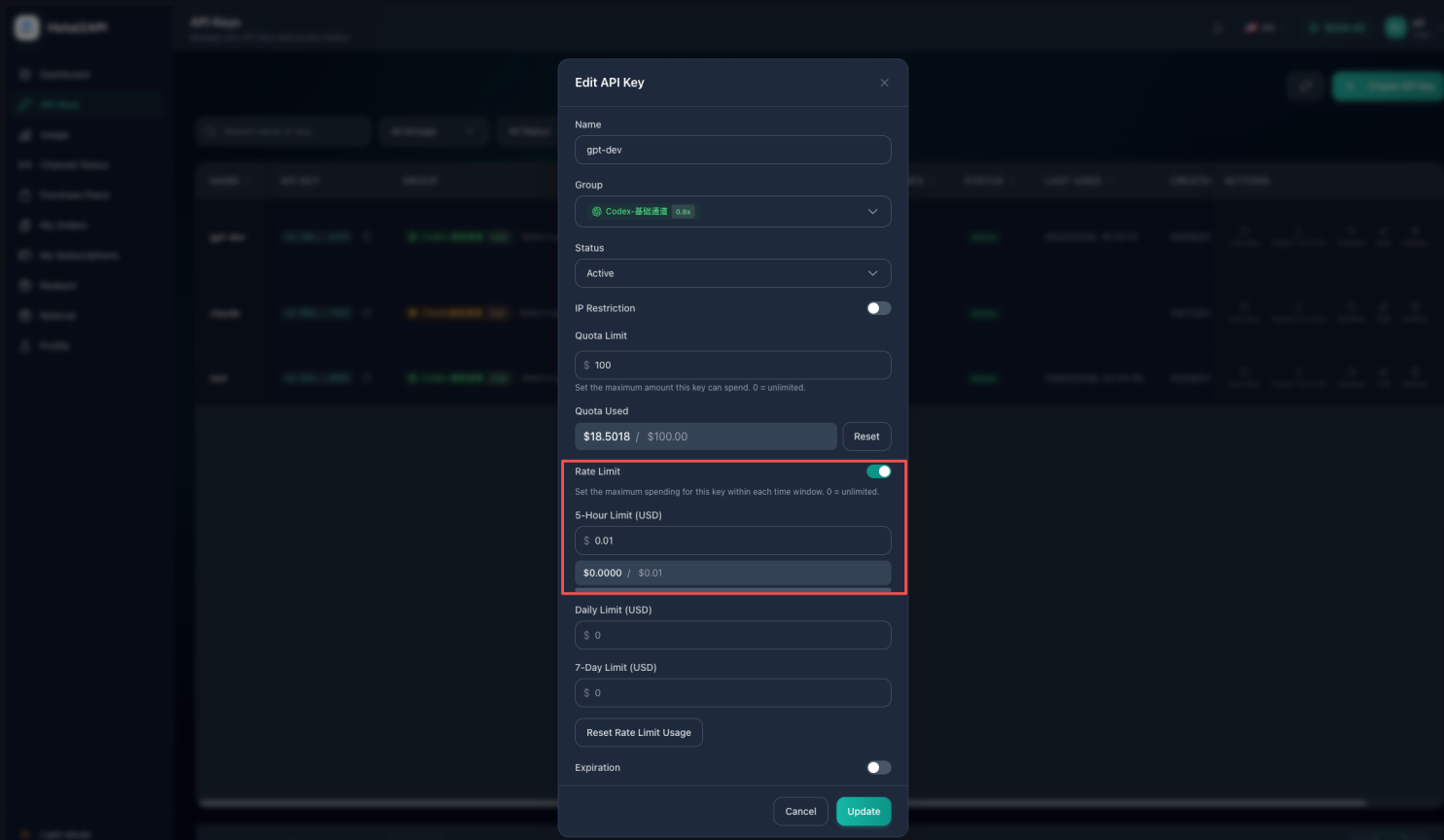
- 1 在飞书中发送 /status，确认当前模型为 hotai/gpt-5.4
- 2 在 Hotai 后台将 API Key 的 rate limit 设为 \$0.01
- 3 在飞书中发送：「你现在用的是什么模型？」→ 确认回复来自 GPT-5.4



					 + Create API Key
Search name or key...					All Groups  All Status 
NAME 	API KEY	GROUP	USAGE	ACTIONS	
gpt-dev	<code>sk-2b6...efe5</code> 	 Codex-基础通道 0.8x Select a group 	Today: \$0.4490 Last 30d: \$18.5465 Quota: \$18.55 / \$100.00 	Use Key  Import to CCS  Disable  Edit  Delete	

实战

准备：设置 Rate Limit 低配额



实战

准备：设置 Rate Limit 低配额

操作指引

- 1 在飞书中发送 /status，确认当前模型为 hotai/gpt-5.4
- 2 在 Hotai 后台将 API Key 的 rate limit 设为 \$0.01
- 3 在飞书中发送：「你现在用的是什么模型？」→ 确认回复来自 GPT-5.4

NAME	API KEY	GROUP	USAGE	RATE LIMIT	EXPIRES	STATUS	ACTIONS
gpt-dev	sk-2b6...efe5	Codex-基础通道 0.8x	Today: \$0.4043 Last 30d: \$18.5018 Quota: \$18.50 / \$100.00	5h \$0.00/\$0.01	Never	Active	Use Key Import to CCS Disable Edit Delete
claude	sk-90d...7443	Claude-基础通道 1.2x	Today: \$0.0000 Last 30d: \$0.0000 Quota: \$0.00 / \$30.00	-	Never	Active	Use Key Import to CCS Disable Edit Delete
test	sk-1be...8d56	Codex-基础通道 0.8x	Today: \$0.0000 Last 30d: \$71.8145	-	Never	Active	Use Key Import to CCS Disable Edit Delete

预期结果

rate limit 生效，当前正在使用 GPT-5.4

触发与验证：让 Fallback 生效，查日志确认

操作指引

- 发送一条消耗 Token 的任务：「帮我写一份 500 字的 AI Agent 市场分析报告」
- Agent 正常回复（Fallback 已静默生效，你可能感觉不到任何差异）
- 注意：此时问 Agent「你用的什么模型？」它会回答 hotai/gpt-5.4——这不是真实情况



你现在用的是什么模型？请告诉我你的模型名称。



我当前使用的模型是 hotai/gpt-5.4，fallback 模型是 volcengine/ark-code-latest。

- 验证：在飞书中发送「帮我查一下 Gateway 日志，最近有没有 fallback 或 429 记录」

预期：日志显示 hotai/gpt-5.4 触发 429 限流（"api key 限额已用完"），已自动 fallback 到 volcengine/ark-code-latest

实战

触发与验证：让 Fallback 生效，查日志确认

```
May 21 16:28:42 openclaw-dev openclaw[958502]: 2026-05-21T16:28:42.224+08:00 [feishu] feishu[default]: Feishu[default] DM from ou_a04cf986b8ee32647460dd46c908ca2c: 你正在用的是什模型？请告诉我你的模型名称。
May 21 16:28:42 openclaw-dev openclaw[958502]: 2026-05-21T16:28:42.329+08:00 [feishu] feishu[default]: dispatching to agent (session=agent:main:main)
May 21 16:28:52 openclaw-dev openclaw[958502]: 2026-05-21T16:28:52.002+08:00 [plugins] plugin tool name conflict (openclaw-lark): feishu_chat
May 21 16:29:06 openclaw-dev openclaw[958502]: 2026-05-21T16:29:06.031+08:00 [agent/embedded] embedded run agent end: runId=ec85784d-a961-4ce6-9128-62bb81bcbcc0 isError=true model=gpt-5.4 provider=hota
i error=⚠️ API rate limit reached. Please try again later. rawError=429 api key 5小时限额已用完
May 21 16:29:09 openclaw-dev openclaw[958502]: 2026-05-21T16:29:09.811+08:00 [agent/embedded] embedded run agent end: runId=ec85784d-a961-4ce6-9128-62bb81bcbcc0 isError=true model=gpt-5.4 provider=hota
i error=⚠️ API rate limit reached. Please try again later. rawError=429 api key 5小时限额已用完
May 21 16:29:15 openclaw-dev openclaw[958502]: 2026-05-21T16:29:15.931+08:00 [agent/embedded] embedded run agent end: runId=ec85784d-a961-4ce6-9128-62bb81bcbcc0 isError=true model=gpt-5.4 provider=hota
i error=⚠️ API rate limit reached. Please try again later. rawError=429 api key 5小时限额已用完
May 21 16:29:25 openclaw-dev openclaw[958502]: 2026-05-21T16:29:25.969+08:00 [agent/embedded] embedded run agent end: runId=ec85784d-a961-4ce6-9128-62bb81bcbcc0 isError=true model=gpt-5.4 provider=hota
i error=⚠️ API rate limit reached. Please try again later. rawError=429 api key 5小时限额已用完
May 21 16:29:26 openclaw-dev openclaw[958502]: 2026-05-21T16:29:26.099+08:00 [agent/embedded] embedded run failover decision: runId=ec85784d-a961-4ce6-9128-62bb81bcbcc0 stage=assistant decision=fallback
k_model reason=rate_limit from=hota/gpt-5.4 profile= rawError=429 api key 5小时限额已用完
May 21 16:29:26 openclaw-dev openclaw[958502]: 2026-05-21T16:29:26.101+08:00 [diagnostic] lane task error: lane=main durationMs=41411 error="FailoverError: ⚠️ API rate limit reached. Please try again la
ter."
May 21 16:29:26 openclaw-dev openclaw[958502]: 2026-05-21T16:29:26.102+08:00 [diagnostic] lane task error: lane=session:agent:main:main durationMs=41413 error="FailoverError: ⚠️ API rate limit reached.
Please try again later."
May 21 16:29:26 openclaw-dev openclaw[958502]: 2026-05-21T16:29:26.107+08:00 [model-fallback/decision] model fallback decision: decision=candidate_failed requested=hota/gpt-5.4 candidate=hota/gpt-5.4
reason=rate_limit next=volcengine/ark-code-latest detail=429 api key 5小时限额已用完
May 21 16:29:29 openclaw-dev openclaw[958502]: 2026-05-21T16:29:29.808+08:00 [plugins] plugin tool name conflict (openclaw-lark): feishu_chat
May 21 16:29:48 openclaw-dev openclaw[958502]: 2026-05-21T16:29:48.033+08:00 [model-fallback/decision] model fallback decision: decision=candidate_succeeded requested=hota/gpt-5.4 candidate=volcengine
/ark-code-latest reason=unknown next=none
```

API Keys

Manage your API keys and access tokens



EN

\$534.38

pj

User



+ Create API Key

Search name or key...

All Groups

All Status

NAME	API KEY	GROUP	USAGE	RATE LIMIT	EXPIRES	STATUS	ACTIONS
gpt-dev	sk-2b6...efe5	Codex-基础通义 0.8x	Today: \$0.4490 Last 30d: \$18.5465 Quota: \$18.55 / \$100.00	5h \$0.04/\$0.01 4h 37m Reset	Never	Active	Use Key Import to CCS Disable Edit Delete

实战

恢复验证：当 Hotai 额度恢复后

场景：在 Hotai 后台将 rate limit 调高（或恢复正常额度）

OpenClaw 定期探测 primary 的可用性，发现恢复后静默切回 GPT-5.4

验证：让 Agent 查 Gateway 日志，确认是否已切回 primary

注意：恢复也是静默的，Agent 不会通知你已切回

API Keys

Manage your API keys and access tokens

API key updated successfully

Search name or key...

All Groups

All Status

NAME	API KEY	GROUP	USAGE	RATE LIMIT	EXPIRES	STATUS	LAST USED	ACTIONS
gpt-dev	sk-2b6...efe5	Codex-基础通晓 0.8x	Today: \$0.4490 Last 30d: \$18.5465 Quota: \$18.55 / \$100.00	5h \$0.04/\$2.00 4h 6m Reset	Never	Active	05/21/2026, 17:00:04	Use Key Import to CCS Disable Edit Delete

实战完整闭环：从熔断到恢复

自动降级保证服务不中断，自动恢复回到最优路径



时间线 (典型流程)



完整闭环总结

自动感知 → 自动降级 → 服务不中断 → 自动探测 → 自动恢复 → 最优路径



用户无感知



服务高可用



成本最优化

实战

恢复验证：当 Hotai 额度恢复后

19:48

查下日志，确认现在用的什么模型

我刚查了近 30 分钟日志，没有看到新的模型调用记录。

当前能确认的是：

- 会话配置里的主模型仍是 hotai/gpt-5.4
- fallback 仍是 volcengine/ark-code-latest
- 最近 30 分钟日志里没有新的 embedded run agent end / failover 记录，所以这段时间没有新的实际调用证据可证明又切了模型

如果你要确认“这一条回复实际走的是哪个模型”，我建议我再查更长时间窗口，或者直接订一条新的调用日志来确认。

Usage: 184k in / 284 out · est \$0.0000

可以，你查看是否已切回 primary 吧，之前是切到 fallback 了

我查了近 6 小时日志，能确认两点：

- 17:00 左右 primary hotai/gpt-5.4 还在报 429 限流
- 随后系统成功切到 fallback volcengine/ark-code-latest

关键日志是：

- candidate_failed requested=hotai/gpt-5.4 ... reason=rate_limit next=volcengine/ark-code-latest
- candidate_succeeded requested=hotai/gpt-5.4 candidate=volcengine/ark-code-latest

但目前还没看到“已切回 primary”的明确成功日志。

也就是说：我现在还不能从日志证明它已经恢复到 hotai/gpt-5.4，已知最后一次有明确成功证据的是 fallback

volcengine/ark-code-latest。

如果你要，我下一步可以继续做一个更直接的确认：触发一条新请求，再立刻抓最新日志，判断这一次实际命中的是 primary 还是 fallback。

Usage: 95k in / 388 out · cache 46k cached / 0 new · est \$0.0000

HEARTBEAT_OK

好呀，你查查

我没法在“这条回复发出之前”再同步抓到它自己的最终命中日志，所以不能在同一瞬间既发消息又验证这条消息实际走了哪个模型。

但结论先给你：

- 从已有日志看，最后一次被明确证实成功的是 fallback: volcengine/ark-code-latest
- 还没有看到切回 primary: hotai/gpt-5.4 的成功证据

如果你要做严格确认，最稳的方法是：你再给我发一条最简单的测试消息，比如“1”，我收到后立刻回一条短消息，然后马上抓最新日志，把那次实际命中的模型告诉你。

Usage: 48k in / 171 out · est \$0.0000

极客时间							
Create API Key							
Search name or key... All Groups All Status							
NAME	GROUP	USAGE	RATE LIMIT	ACTIONS			
gpt-dev	Codex-基础通信 0.8x	Today: \$2.1273 Last 30d: \$20.2247 Quota: \$20.22 / \$100.00	5h \$1.72/\$2.00 1h 26m Reset	Use Key	Import to CCS	Disable	Edit Delete
claude	Claude基础通信 1.2x	Today: \$0.0000 Last 30d: \$0.0000 Quota: \$0.00 / \$30.00	-	Use Key	Import to CCS	Disable	Edit Delete
test	Codex-基础通信 0.8x	Today: \$0.0000 Last 30d: \$71.8145	-	Use Key	Import to CCS	Disable	Edit Delete

API KEY	BILLING MODE	TOKENS	COST	FIRST TOKEN	DURATION	TIME	USER-AGENT
gpt-dev	Token	↓ 48,114 ↑ 171	\$0.098280	2.07s	5.58s	05/21/2026, 19:50:41	OpenAI/JS 6.26.0
gpt-dev	Token	↓ 1,513 ↑ 241 46.1K	\$0.015134	1.69s	6.24s	05/21/2026, 19:49:37	OpenAI/JS 6.26.0
gpt-dev	Token	↓ 46,786 ↑ 30	\$0.093932	2.03s	2.65s	05/21/2026, 19:49:30	OpenAI/JS 6.26.0
gpt-dev	Token	↓ 46,626 ↑ 117	\$0.094656	2.23s	4.20s	05/21/2026, 19:49:24	OpenAI/JS 6.26.0
gpt-dev	Token	↓ 46,179 ↑ 149	\$0.094146	1.74s	4.57s	05/21/2026, 19:48:35	OpenAI/JS 6.26.0
gpt-dev	Token	↓ 46,049 ↑ 31	\$0.092470	2.65s	3.09s	05/21/2026, 19:48:29	OpenAI/JS 6.26.0
gpt-dev	Token	↓ 45,915 ↑ 90	\$0.092910	1.79s	3.46s	05/21/2026, 19:48:24	OpenAI/JS 6.26.0
gpt-dev	Token	↓ 45,716 ↑ 14	\$0.091600	2.47s	2.67s	05/21/2026, 19:48:20	OpenAI/JS 6.26.0

踩坑速查表：混合路由常见问题

现象	原因	解决方案
改了 openclaw.json 但没生效	models.mode 设为 merge, Agent 本地缓存了旧配置	改为 mode: "replace"
改了 .env 但没生效	环境变量在 Gateway 启动时解析	在飞书中发送「重启 OpenClaw Gateway」
问 Agent "你用的什么模型" 回答不准	Fallback 是 Gateway 层静默切换 Agent 无感知	让 Agent 查 Gateway 日志 确认实际使用的模型
/status 显示的模型不是 primary	没有配置 agents.defaults. model.primary	在 openclaw.json 中添加 primary 保存后热加载生效
Missing auth	.env 中的变量名与 openclaw.json 中的 \${VAR_NAME} 不匹配	检查两边的变量名拼写完全一致
baseUrl 报 origin not allowed	火山引擎 baseUrl 路径错误	必须使用 /api/coding/v3 (第3节踩坑实录)

第五章

总结与思考

本节核心收获与课后作业

本节核心收获

① 成本真相

同一 Agent, Coding Plan Pro ¥200 封顶 vs GPT-5.4 ¥930+, Token 大头是 System Prompt 和对话历史

② 旗舰选型

GPT-5.4 性价比优先 (日常默认), Opus 4.6 精度优先 (关键任务手动切), 多花 70% 换 50%+ 返工减少

③ 双引擎架构

中转站 (按量旗舰) + 火山引擎 Coding Plan (包月国产), 一份 openclaw.json 管理所有模型

④ Fallback 内置能力

配置 `agents.defaults.model.fallbacks`, API 报错时自动降级, 恢复后自动升回

⑤ 三级成本防线

提供商配额硬熔断 → Fallback 自动降级 → /usage 日常监控。成本控制不是全用便宜模型, 而是把钱花在高价值的任务上。

课后作业

① 必做：双 Provider 配置

在自己服务器上完成双 Provider 配置（Hotai + 火山引擎）

在飞书中发送 /status 截图确认当前模型为 Primary 设置的模型

② 必做：触发一次 Fallback

在 Hotai 后台设置低配额，触发一次 Fallback。在飞书中截图，验证模型切换通知